



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe de Arquitectura de Software

Sistema Analizador de Rendimiento de Consultas (Query Analyzer)

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-29	Versión inicial
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Actualización factual y formato institucional
1.2	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-07-04	Ampliación con vistas 4+1, despliegue y atributos de calidad

ÍNDICE GENERAL

1. [INTRODUCCIÓN](#)
2. [Propósito \(Diagrama 4+1\)](#)
3. [Alcance](#)
4. [Definición, siglas y abreviaturas](#)
5. [Organización del documento](#)
6. [OBJETIVOS Y RESTRICCIONES ARQUITECTÓNICAS](#)
7. [Requerimientos Funcionales](#)
8. [Requerimientos No Funcionales - Atributos de Calidad](#)
9. [REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA](#)
10. [Vista de Caso de uso](#)
11. [Vista Lógica](#)
12. [Vista de Implementación \(vista de desarrollo\)](#)
13. [Vista de procesos](#)
14. [Vista de Despliegue \(vista física\)](#)
15. [ATRIBUTOS DE CALIDAD DEL SOFTWARE](#)
16. [Escenario de Funcionalidad](#)
17. [Escenario de Usabilidad](#)
18. [Escenario de confiabilidad](#)
19. [Escenario de rendimiento](#)
20. [Escenario de mantenibilidad](#)
21. [Otros Escenarios](#)

1. INTRODUCCIÓN

Query Analyzer es una herramienta local y extensible para analizar planes de ejecución, métricas observables y evidencia técnica de consultas en múltiples motores de bases de datos. La solución ofrece varias entradas de uso: CLI, TUI, API REST local, servidor MCP y extensión para Visual Studio Code. Todas reutilizan un núcleo común basado en modelos Pydantic, registro de adaptadores y reportes factuales.

Este documento describe la arquitectura de software del sistema usando el modelo de vistas 4+1. La arquitectura combina capas, patrón Adapter, contratos de datos y separación explícita entre evidencia factual e interpretación opcional mediante IA.

1.1. Propósito (Diagrama 4+1)

El propósito del documento es comunicar cómo se organiza Query Analyzer para cumplir sus requisitos funcionales, atributos de calidad y restricciones de operación local. El modelo 4+1 permite describir la arquitectura desde cinco perspectivas complementarias: casos de uso, lógica, implementación, procesos y despliegue.

```
flowchart TD
    UC["Vista de casos de uso<br/>actores y escenarios"]
    LOG["Vista lógica<br/>modelos, clases y subsistemas"]
    IMP["Vista de implementación<br/>paquetes y componentes"]
    PROC["Vista de procesos<br/>flujos de ejecución"]
    DEP["Vista de despliegue<br/>nodos físicos y distribución"]
    ARQ["Arquitectura Query Analyzer<br/>modelo 4+1"]

    UC --> ARQ
    LOG --> ARQ
    IMP --> ARQ
    PROC --> ARQ
    DEP --> ARQ
```

1.2. Alcance

El informe cubre la arquitectura de la versión documentada de Query Analyzer, incluyendo:

- administración segura de perfiles;
- diagnóstico de conexiones;
- ejecución de `EXPLAIN` o mecanismo equivalente;
- normalización parcial mediante `PlanNode` ;
- construcción de `QueryAnalysisReport` ;
- conservación de `raw_plan` y métricas específicas;
- exportación JSON/Markdown;
- CLI, TUI, API REST local, MCP y extensión VS Code;
- análisis opcional con IA mediante `AIAnalysisResult` ;
- documentación, pruebas y distribución multiplataforma.

Quedan fuera del alcance arquitectónico las modificaciones automáticas de consultas, índices o esquemas, el monitoreo continuo de producción y cualquier score universal de calidad entre motores.

1.3. Definición, siglas y abreviaturas

Término	Definición
API	Interfaz HTTP local implementada con FastAPI
CLI	Interfaz de línea de comandos
TUI	Interfaz terminal interactiva implementada con Textual
MCP	Model Context Protocol para integración con agentes
VS Code	Visual Studio Code, editor integrado mediante extensión
Adapter	Componente que encapsula diferencias de un motor de base de datos
AdapterRegistry	Fábrica y catálogo de adaptadores registrados
ConnectionConfig	Modelo de configuración validada para conectarse a un motor
PlanNode	Nodo recursivo y agnóstico para representar planes normalizados
QueryAnalysisReport	Reporte factual con consulta, motor, métricas, plan y fecha
AIAnalysisResult	Interpretación opcional generada por IA, separada del reporte factual
raw_plan	Plan original entregado por el motor

1.4. Organización del documento

El documento se organiza en cuatro secciones principales. La primera define propósito, alcance y terminología. La segunda resume objetivos y restricciones arquitectónicas. La tercera presenta las vistas 4+1 del sistema mediante diagramas. La cuarta describe escenarios de calidad relacionados con funcionalidad, usabilidad, confiabilidad, rendimiento, mantenibilidad y seguridad.

2. OBJETIVOS Y RESTRICCIONES ARQUITECTÓNICAS

La arquitectura de Query Analyzer responde a una decisión central: el sistema debe presentar evidencia real de motores heterogéneos sin inventar métricas ni mezclar interpretación con datos factuales. Por ello, el diseño favorece contratos explícitos, adaptadores desacoplados, validación de entradas, sanitización de secretos y reutilización del núcleo por todas las interfaces.

2.1.1. Requerimientos Funcionales

ID	Requerimiento funcional arquitectónico	Soporte arquitectónico
RF-01	Registrar y crear adaptadores por motor	<code>AdapterRegistry</code> y <code>BaseAdapter</code>
RF-02	Validar conexiones por motor	<code>ConnectionConfig</code> y validadores Pydantic
RF-03	Administrar perfiles locales	<code>query_analyzer.config</code> y almacenamiento YAML cifrado
RF-04	Diagnosticar conexiones	<code>ConnectionDiagnostics</code> y adaptadores
RF-05	Ejecutar <code>EXPLAIN</code> o equivalente	Adaptadores SQL, NoSQL, grafos y series de tiempo
RF-06	Construir reportes factuales	<code>QueryAnalysisReport</code>
RF-07	Normalizar planes jerárquicos	<code>PlanNode</code> y parsers por motor
RF-08	Conservar plan original y métricas	Campos <code>raw_plan</code> y <code>metrics</code>
RF-09	Mostrar resultados en CLI y TUI	Paquetes <code>cli</code> y <code>tui</code>
RF-10	Exponer API REST	Paquete <code>api</code> bajo <code>/api/v1/analyzer</code>
RF-11	Exponer herramienta MCP	<code>query_analyzer.mcp_server</code>
RF-12	Analizar desde VS Code	Extensión <code>integrations/vscode-query-analyzer</code>

ID	Requerimiento funcional arquitectónico	Soporte arquitectónico
RF-13	Exportar reportes	ReportSerializer
RF-14	Solicitar IA opcional	AIAnalyzer y AIAnalysisResult

2.1.2. Requerimientos No Funcionales - Atributos de Calidad

Atributo	Requerimiento	Decisión arquitectónica
Funcionalidad	Soportar 13 motores registrados	Catálogo explícito de adaptadores
Seguridad	No exponer secretos	Cifrado, SecretStr, sanitización y API sin persistencia de credenciales
Confiabilidad	Un fallo de IA no invalida el reporte factual	IA opcional y separada
Rendimiento	Evitar trabajo innecesario en la capa común	Delegación por adaptador y ejecución local
Usabilidad	Ofrecer varios canales de uso	CLI, TUI, API, MCP y VS Code
Mantenibilidad	Agregar motores sin reescribir interfaces	Patrón Adapter y contratos comunes
Portabilidad	Ejecutar en entornos locales y distribuidos por binario	Python, PyInstaller, JReleaser, VSIX
Observabilidad académica	Publicar documentación y evidencias	GitHub Pages y reportes de calidad

Restricciones arquitectónicas

- Usar uv como flujo de ejecución y dependencias.
- Mantener el servidor API en 127.0.0.1 por defecto.
- No persistir credenciales enviadas por API.
- No calcular score universal de calidad.
- No reemplazar métricas ausentes por cero.
- Separar pruebas unitarias de pruebas de integración con Docker.
- Mantener QueryAnalysisReport como contrato común entre interfaces.

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

3.1. Vista de Caso de uso

La vista de caso de uso identifica los actores que interactúan con Query Analyzer y los servicios principales que la arquitectura debe soportar.

3.1.1. Diagramas de Casos de uso

```
flowchart LR
    EST["Estudiante / Desarrollador / DBA"]
    DOC["Docente / evaluador"]
    HTTP["Cliente HTTP"]
    AG["Agente MCP"]
    VSC["Usuario VS Code"]
    SYS["Query Analyzer"]

    EST --> UC1["Administrar perfiles"]
    EST --> UC2["Diagnosticar conexión"]
    EST --> UC3["Analizar consulta"]
    EST --> UC4["Exportar reporte"]
    EST --> UC5["Consultar historial"]
    EST --> UC6["Solicitar IA opcional"]
    DOC --> UC7["Revisar documentación y evidencias"]
    HTTP --> UC8["Invocar API REST"]
    AG --> UC9["Invocar analyze_query"]
    VSC --> UC10["Analizar selección SQL"]

    UC1 --> SYS
    UC2 --> SYS
    UC3 --> SYS
    UC4 --> SYS
    UC5 --> SYS
    UC6 --> SYS
    UC7 --> SYS
    UC8 --> SYS
    UC9 --> SYS
    UC10 --> SYS
```

3.2. Vista Lógica

La vista lógica muestra los subsistemas de dominio y sus relaciones. El núcleo de la solución es independiente de las interfaces y se apoya en modelos comunes.

3.2.1. Diagrama de Subsistemas (paquetes)

```
flowchart TD
    CLI["query_analyzer.cli"]
    TUI["query_analyzer.tui"]
    API["query_analyzer.api"]
    MCP["query_analyzer.mcp_server"]
```



```
CFG["query_analyzer.config"]
CORE["query_analyzer.core"]
ADP["query_analyzer.adapters"]
SQL["adapters.sql"]
NOSQL["adapters.nosql"]
GRAPH["adapters.graph"]
TS["adapters.timeseries"]
VSC["integrations/vscode-query-analyzer"]
```

```
CLI --> CFG
CLI --> ADP
TUI --> CFG
TUI --> ADP
API --> ADP
API --> CORE
MCP --> CFG
MCP --> ADP
VSC --> API
CORE --> ADP
ADP --> SQL
ADP --> NOSQL
ADP --> GRAPH
ADP --> TS
```

3.2.2. Diagrama de Secuencia (vista de diseño)

La vista de diseño presenta un diagrama de secuencia por cada caso de uso principal. De esta forma se evita ocultar diferencias entre la administración de perfiles, el análisis factual, la exportación, la IA opcional y las integraciones externas.

CU-01. Administrar perfil de conexión

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Config as ConfigManager
    participant Crypto as Servicio de cifrado

    Usuario->>Interfaz: Solicita crear/editar/eliminar perfil
    Interfaz->>Usuario: Solicita motor y datos de conexión
    Usuario->>Interfaz: Ingresa ConnectionConfig
    Interfaz->>Config: Validar motor, puerto y credenciales
    alt Configuración válida
        Config->>Crypto: Cifrar credenciales
        Crypto-->>Config: Secretos cifrados
        Config->>Config: Persistir perfil local
        Config-->>Interfaz: Perfil actualizado
        Interfaz-->>Usuario: Confirmación segura
    else Configuración inválida
        Config-->>Interfaz: Error de validación
        Interfaz-->>Usuario: Mensaje sin secretos
    end
end
```

CU-02. Diagnosticar conexión

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI/API
```

```
participant Config as ConfigManager
participant Diagnostico as ConnectionDiagnostics
participant Registro as AdapterRegistry
participant Adaptador as BaseAdapter
participant Motor as Motor de BD
```

```
Usuario->>Interfaz: Solicita diagnóstico
Interfaz->>Config: Obtener perfil o conexión
Config-->>Interfaz: ConnectionConfig validado
Interfaz->>Diagnostico: Ejecutar comprobaciones
Diagnostico->>Registro: Verificar motor registrado
Registro-->>Diagnostico: Motor soportado
Diagnostico->>Adaptador: test_connection()
Adaptador->>Motor: DNS/TCP/autenticación/operación
Motor-->>Adaptador: Resultado técnico
Adaptador-->>Diagnostico: Estado de conexión
Diagnostico-->>Interfaz: Diagnóstico sanitizado
Interfaz-->>Usuario: Estado, duración y detalle seguro
```

CU-03. Analizar consulta

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Config as ConfigManager
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport
    participant Historial as HistoryManager

    Usuario->>Interfaz: Ingresa consulta y perfil
    Interfaz->>Config: Resolver perfil seleccionado
    Config-->>Interfaz: ConnectionConfig
    Interfaz->>Registro: create(engine, config)
    Registro-->>Interfaz: Adaptador concreto
    Interfaz->>Adaptador: connect()
    Adaptador->>Motor: EXPLAIN / PROFILE / equivalente
    Motor-->>Adaptador: Plan y métricas disponibles
    Adaptador->>Reporte: Construir reporte factual
    Reporte-->>Interfaz: QueryAnalysisReport
    Interfaz->>Historial: Guardar si corresponde
    Interfaz->>Adaptador: disconnect()
    Interfaz-->>Usuario: Mostrar plan, métricas y resumen
```

CU-04. Exportar reporte

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Reporte as QueryAnalysisReport
    participant Serializador as ReportSerializer
    participant Archivo as Sistema de archivos

    Usuario->>Interfaz: Solicita exportar reporte
    Interfaz->>Reporte: Obtener reporte actual
    Reporte-->>Interfaz: Evidencia factual
    Interfaz->>Serializador: to_json() o to_markdown()
    Serializador-->>Interfaz: Contenido serializado
    Interfaz->>Archivo: Escribir destino
```

```

alt Escritura correcta
  Archivo-->>Interfaz: Archivo creado
  Interfaz-->>Usuario: Ruta de exportación
else Error de escritura
  Archivo-->>Interfaz: Error controlado
  Interfaz-->>Usuario: Mensaje sin perder reporte
end

```

CU-05. Solicitar interpretación IA opcional

```

sequenceDiagram
  actor Usuario
  participant Interfaz as CLI/TUI/API
  participant Reporte as QueryAnalysisReport
  participant IA as AIAalyzer
  participant ResultadoIA as AIAalysisResult

  Usuario->>Interfaz: Solicita IA opcional
  Interfaz->>Reporte: Obtener plan, consulta y motor
  Reporte-->>Interfaz: Evidencia factual
  Interfaz->>IA: analyze(plan_json, query, engine)
  alt IA configurada y disponible
    IA-->>ResultadoIA: Resumen y recomendaciones
    ResultadoIA-->>Interfaz: Interpretación separada
    Interfaz-->>Usuario: Muestra IA sin alterar métricas
  else IA ausente o con error
    IA-->>Interfaz: Sin resultado IA
    Interfaz-->>Usuario: Mantiene reporte factual
  end
end

```

CU-06. Usar API REST

```

sequenceDiagram
  actor Cliente as Cliente HTTP
  participant API as API REST
  participant Registro as AdapterRegistry
  participant Adaptador as BaseAdapter
  participant Motor as Motor de BD
  participant Reporte as QueryAnalysisReport

  Cliente->>API: POST /api/v1/analyzer/explain
  API->>API: Validar request y ConnectionConfig
  API->>Registro: create(engine, connection)
  Registro-->>API: Adaptador concreto
  API->>Adaptador: connect()
  Adaptador->>Motor: EXPLAIN / equivalente
  Motor-->>Adaptador: Plan y métricas
  Adaptador->>Reporte: Construir reporte
  Reporte-->>API: Resultado estructurado
  API->>Adaptador: disconnect()
  API-->>Cliente: JSON sin persistir credenciales

```

CU-07. Usar herramienta MCP

```

sequenceDiagram
  actor Agente as Agente MCP
  participant MCP as MCP Server

```

```

participant Config as ConfigManager
participant Registro as AdapterRegistry
participant Adaptador as BaseAdapter
participant Motor as Motor de BD
participant Reporte as QueryAnalysisReport

Agente->>MCP: analyze_query(query, profile)
MCP->>Config: Resolver perfil indicado o default
Config-->>MCP: ConnectionConfig
MCP->>Registro: create(engine, config)
Registro-->>MCP: Adaptador concreto
MCP->>Adaptador: connect()
Adaptador->>Motor: EXPLAIN / equivalente
Motor-->>Adaptador: Plan y métricas
Adaptador->>Reporte: Construir reporte factual
Reporte-->>MCP: Resultado estructurado
MCP->>Adaptador: disconnect()
MCP-->>Agente: Respuesta de herramienta

```

CU-08. Analizar desde Visual Studio Code

```

sequenceDiagram
    actor Usuario
    participant VSCode as VS Code Extension
    participant Backend as Backend local
    participant API as API REST
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport

    Usuario->>VSCode: Selecciona consulta y ejecuta comando
    VSCode->>Backend: Verificar/iniciar backend
    Backend-->>VSCode: Backend disponible
    VSCode->>API: Enviar consulta y perfil
    API->>Registro: create(engine, config)
    Registro-->>API: Adaptador concreto
    API->>Adaptador: connect()
    Adaptador->>Motor: EXPLAIN / equivalente
    Motor-->>Adaptador: Plan y métricas
    Adaptador->>Reporte: Construir reporte factual
    Reporte-->>API: Resultado estructurado
    API->>Adaptador: disconnect()
    API-->>VSCode: Reporte para vista del editor
    VSCode-->>Usuario: Muestra análisis

```

3.2.3. Diagrama de Colaboración (vista de diseño)

```

flowchart LR
    U["Usuario"] --> I["Interfaz"]
    I --> C["ConfigManager"]
    I --> R["AdapterRegistry"]
    R --> A["Adaptador concreto"]
    A --> DB["Motor de BD"]
    A --> P["Parser / normalizador"]
    P --> REP["QueryAnalysisReport"]
    REP --> S["ReportSerializer"]
    REP --> H["HistoryManager"]

```

```
REP --> AI["AIAnalyzer opcional"]
S --> OUT["JSON / Markdown / Pantalla"]
```

3.2.4. Diagrama de Objetos

```
classDiagram
    class profile_local {
        name = "local-postgres"
        engine = "postgresql"
        host = "localhost"
        database = "query_analyzer"
    }

    class connection_config {
        engine = "postgresql"
        port = 5432
        username = "qa"
        password = "****"
    }

    class adapter_instance {
        type = "PostgreSQLAdapter"
        connected = false
    }

    class report {
        engine = "postgresql"
        query = "SELECT ..."
        execution_time_ms = 12.4
        plan_summary = "Seq Scan ..."
    }

    class plan_node {
        node_type = "Seq Scan"
        actual_rows = 95
    }

    profile_local --> connection_config
    connection_config --> adapter_instance
    adapter_instance --> report
    report --> plan_node
```

3.2.5. Diagrama de Clases

```
classDiagram
    class ConnectionConfig {
        +str engine
        +str host
        +int port
        +str database
        +str username
        +str password
        +dict extra
    }

    class AdapterRegistry {
        +register(engine)
        +create(engine, config)
        +list_engines()
        +is_registered(engine)
    }
```

```

}

class BaseAdapter {
    <<abstract>>
    +connect()
    +disconnect()
    +test_connection()
    +execute_explain(query) QueryAnalysisReport
    +get_slow_queries()
    +get_metrics()
    +get_engine_info()
}

class PlanNode {
    +str node_type
    +float cost
    +int estimated_rows
    +int actual_rows
    +float actual_time_ms
    +list children
    +dict properties
}

class QueryAnalysisReport {
    +str engine
    +str query
    +float execution_time_ms
    +PlanNode plan_tree
    +str plan_summary
    +dict raw_plan
    +dict metrics
    +datetime analyzed_at
    +AIAnalysisResult ai_analysis
}

class AIAnalysisResult {
    +str summary
    +list observations
    +list recommendations
    +str suggested_query
    +str raw_response
}

class ReportSerializer {
    +to_json(report)
    +from_json(text)
    +to_markdown(report)
    +to_dict(report)
}

AdapterRegistry --> BaseAdapter
BaseAdapter --> ConnectionConfig
BaseAdapter --> QueryAnalysisReport
QueryAnalysisReport --> PlanNode
QueryAnalysisReport --> AIAnalysisResult
ReportSerializer --> QueryAnalysisReport

```

3.2.6. Diagrama de Base de datos (relacional o no relacional)

Query Analyzer no usa una base de datos central obligatoria. Persiste configuración e historial de forma local mediante archivos YAML/JSON, y se conecta a motores externos para obtener

evidencia de análisis. Por ello, el diagrama representa persistencia local no relacional y motores externos.

```
erDiagram
    APP_CONFIG ||--o{ PROFILE_CONFIG : contains
    APP_CONFIG ||--|| APP_DEFAULTS : defines
    PROFILE_CONFIG ||--o{ HISTORY_ENTRY : produces
    HISTORY_ENTRY ||--|| QUERY_ANALYSIS_REPORT : stores
    QUERY_ANALYSIS_REPORT ||--o{ PLAN_NODE : contains
    QUERY_ANALYSIS_REPORT ||--o| AI_ANALYSIS_RESULT : optionally_has

    APP_CONFIG {
        string active_profile
        string config_path
    }
    PROFILE_CONFIG {
        string name
        string engine
        string host
        int port
        string database
        string encrypted_password
    }
    HISTORY_ENTRY {
        string profile_name
        datetime created_at
        string notes
    }
    QUERY_ANALYSIS_REPORT {
        string engine
        string query
        float execution_time_ms
        string plan_summary
        datetime analyzed_at
    }
    PLAN_NODE {
        string node_type
        float cost
        int estimated_rows
        int actual_rows
    }
    AI_ANALYSIS_RESULT {
        string summary
        string recommendations
    }
```

3.3. Vista de Implementación (vista de desarrollo)

La vista de implementación describe la organización del código y los componentes desplegados. El repositorio mantiene paquetes separados por responsabilidad y adaptadores agrupados por familia de motor.

3.3.1. Diagrama de arquitectura software (paquetes)

```
flowchart TD
    ROOT["Repositorio Query Analyzer"]
    QA["query_analyzer/"]
    DOCS["docs/"]
    TESTS["tests/"]
```

```

SCRIPTS["scripts/"]
INTEGRATIONS["integrations/"]
DOCKER["docker/"]

ROOT --> QA
ROOT --> DOCS
ROOT --> TESTS
ROOT --> SCRIPTS
ROOT --> INTEGRATIONS
ROOT --> DOCKER

QA --> ADP["adapters/"]
QA --> CFG["config/"]
QA --> CORE["core/"]
QA --> CLI["cli/"]
QA --> TUI["tui/"]
QA --> API["api/"]
QA --> MCP["mcp_server.py"]
INTEGRATIONS --> VSC["vscode-query-analyzer/"]
INTEGRATIONS --> SKILL["skills/query-analyzer-mcp/"]

```

3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)

```

flowchart TD
    USER["Usuario"]
    AGENT["Agente MCP"]
    HTTP["Cliente HTTP"]
    EDITOR["VS Code"]

    CLI["CLI Typer/Rich"]
    TUI["TUI Textual"]
    API["API FastAPI"]
    MCP["Servidor MCP"]
    EXT["Extensión VS Code"]
    CORE["Modelos y servicios"]
    CFG["Gestor de perfiles"]
    REG["AdapterRegistry"]
    AI["AIAalyzer opcional"]
    SER["ReportSerializer"]
    HIST["Historial local"]
    ADP["Adaptadores por motor"]
    DB["Motores externos"]

    USER --> CLI
    USER --> TUI
    HTTP --> API
    AGENT --> MCP
    EDITOR --> EXT
    EXT --> API
    CLI --> CORE
    TUI --> CORE
    API --> CORE
    MCP --> CORE
    CORE --> CFG
    CORE --> REG
    CORE --> AI
    CORE --> SER
    CORE --> HIST
    REG --> ADP
    ADP --> DB

```


3.4. Vista de procesos

La vista de procesos presenta el flujo operativo principal. La arquitectura debe asegurar validación, conexión, ejecución, reporte, cierre de recursos y respuesta segura.

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)

```
flowchart TD
    A["Inicio"] --> B["Seleccionar interfaz"]
    B --> C["Resolver perfil o conexión"]
    C --> D["Validar ConnectionConfig"]
    D --> E{"¿Configuración válida?"}
    E -- "No" --> F["Mostrar error sanitizado"]
    F --> Z["Fin"]
    E -- "Sí" --> G["Crear adaptador"]
    G --> H["Abrir conexión"]
    H --> I["Ejecutar EXPLAIN o equivalente"]
    I --> J["Parsear salida del motor"]
    J --> K["Construir PlanNode si aplica"]
    K --> L["Conservar raw_plan y metrics"]
    L --> M["Construir QueryAnalysisReport"]
    M --> N{"¿IA configurada?"}
    N -- "Sí" --> O["Crear AIAnalysisResult separado"]
    N -- "No" --> P["Continuar sin IA"]
    O --> Q["Presentar/exportar reporte"]
    P --> Q
    Q --> R["Guardar historial si corresponde"]
    R --> S["Cerrar conexión"]
    S --> Z
```

3.5. Vista de Despliegue (vista física)

La vista de despliegue muestra los nodos físicos y entornos donde opera Query Analyzer. La aplicación puede ejecutarse desde código fuente, binario distribuido, API local, extensión VS Code o servidor MCP.

3.5.1. Diagrama de despliegue

```
flowchart TD
    DEV["Equipo del usuario<br/>Windows / Linux / macOS"]
    BIN["Binario qa / uv run"]
    LOCALCFG["Archivos locales<br/>perfiles cifrados e historial"]
    API["API local<br/>127.0.0.1"]
    VSC["VS Code + extensión"]
    MCP["Cliente agente + MCP stdio"]
    DOCKER["Docker Compose opcional"]
    DBS["Motores de BD<br/>locales o remotos"]
    GITHUB["GitHub Actions / Releases / Pages"]
    PKG["Homebrew / Scoop / Snap / VSIX"]

    DEV --> BIN
    DEV --> API
    DEV --> VSC
    DEV --> MCP
    BIN --> LOCALCFG
    API --> LOCALCFG
```

```
VSC --> API
MCP --> BIN
BIN --> DBS
API --> DBS
DOCKER --> DBS
GITHUB --> PKG
GITHUB --> DOCS["Sitio documental Pages"]
```

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

Los atributos de calidad se expresan como escenarios verificables. Cada escenario indica estímulo, ambiente, respuesta esperada y medida de aceptación.

Escenario de Funcionalidad

Elemento	Descripción
Estímulo	Un usuario solicita analizar una consulta en un motor soportado
Ambiente	CLI, TUI, API, MCP o VS Code con conexión válida
Respuesta	El sistema crea el adaptador, ejecuta el mecanismo de análisis y devuelve <code>QueryAnalysisReport</code>
Medida	El reporte contiene motor, consulta, tiempo mayor que cero, resumen, fecha, métricas disponibles y plan original si existe

Escenario de Usabilidad

Elemento	Descripción
Estímulo	Un estudiante necesita diagnosticar una consulta sin conocer todos los comandos del motor
Ambiente	Equipo local con perfil configurado
Respuesta	El sistema guía la selección de perfil, ejecuta el análisis y presenta resultados legibles
Medida	El usuario obtiene el reporte desde CLI/TUI o VS Code sin manipular directamente el driver

Escenario de confiabilidad

Elemento	Descripción
Estímulo	El proveedor de IA no está configurado o falla durante el análisis
Ambiente	Análisis factual ya ejecutado contra un motor
Respuesta	El sistema omite <code>AIAnalysisResult</code> y conserva el reporte factual
Medida	La operación no falla por ausencia de IA y no altera métricas reales

Escenario de rendimiento

Elemento	Descripción
Estímulo	El usuario ejecuta un análisis desde CLI o API local
Ambiente	Motor accesible y consulta válida
Respuesta	El sistema delega el trabajo al adaptador sin pasar por capas innecesarias
Medida	El tiempo reportado corresponde al análisis observado y el cierre de conexión se ejecuta al finalizar

Escenario de mantenibilidad

Elemento	Descripción
Estímulo	Se requiere añadir un nuevo motor soportado
Ambiente	Código fuente del proyecto y contrato <code>BaseAdapter</code>
Respuesta	Se implementa un adaptador nuevo, se registra en <code>AdapterRegistry</code> y se agregan pruebas
Medida	Las interfaces superiores no necesitan cambiar para crear el adaptador y consumir <code>QueryAnalysisReport</code>

Otros Escenarios

Seguridad

Elemento	Descripción
Estímulo	Ocurre un error de conexión con URI, token o contraseña
Ambiente	CLI, TUI o API
Respuesta	El sistema devuelve mensaje comprensible y sanitizado
Medida	No aparecen contraseñas, tokens, API keys ni cabeceras Bearer en respuestas públicas

Portabilidad

Elemento	Descripción
Estímulo	Un usuario instala Query Analyzer en Windows, Linux o macOS

Elemento	Descripción
Ambiente	Binario, gestor de paquetes o código fuente con <code>uv</code>
Respuesta	El sistema ejecuta los mismos comandos principales y conserva el contrato de reporte
Medida	CLI, API y extensión pueden operar sin cambiar el diseño de adaptadores

Interoperabilidad

Elemento	Descripción
Estímulo	Una herramienta externa necesita consumir análisis de consultas
Ambiente	API REST local o MCP por stdio
Respuesta	El sistema entrega datos estructurados sin depender de la presentación CLI/TUI
Medida	Clientes HTTP y agentes MCP reciben reportes compatibles con los modelos documentados

Trazabilidad documental

Elemento	Descripción
Estímulo	El docente o equipo revisa evidencias del proyecto
Ambiente	GitHub Pages generado desde Markdown
Respuesta	La documentación institucional, reportes y enlaces se renderizan de forma navegable
Medida	<code>scripts/build-pages.py</code> genera el sitio sin Markdown crudo en portadas ni enlaces rotos relevantes

